Examination Result and Grace-Mark Validation – Complete Python Solution

Problem Statement:

Develop a Python program to process student marks across multiple subjects, check subject-wise pass criteria, apply a specified grace-mark policy, and generate final examination classifications without exceeding maximum subject marks.

The solution uses only your assigned Python syllabus concepts.

1. Computational Thinking

Input:

- Student name and roll number.
- Number of subjects.
- Subject names.
- Marks obtained in each subject.
- Maximum marks for each subject.
- Grace-mark limit.

Processing:

1. Read student details.
2. Read marks for all subjects.
3. Check whether marks are within the maximum limit.
4. Identify failed subjects.
5. Apply grace marks according to the given policy.
6. Ensure marks do not exceed maximum marks.
7. Calculate total and percentage.
8. Generate final result and classification.

Output:

- Student details.
- Subject-wise marks.
- Grace marks awarded.

- Final marks.
- Total marks and percentage.
- Pass or fail status.
- Final classification.

2. Grace-Mark Policy

For this example, assume:

Condition	Policy
Maximum grace marks	5
Subject pass marks	40
Grace eligibility	Student must be short by 1–5 marks
Number of subjects eligible	Maximum 1 subject
Maximum subject marks	100
Marks after grace	Cannot exceed 100

Example: If a student scores 37 out of 100, the student needs 3 grace marks to reach 40 and pass.

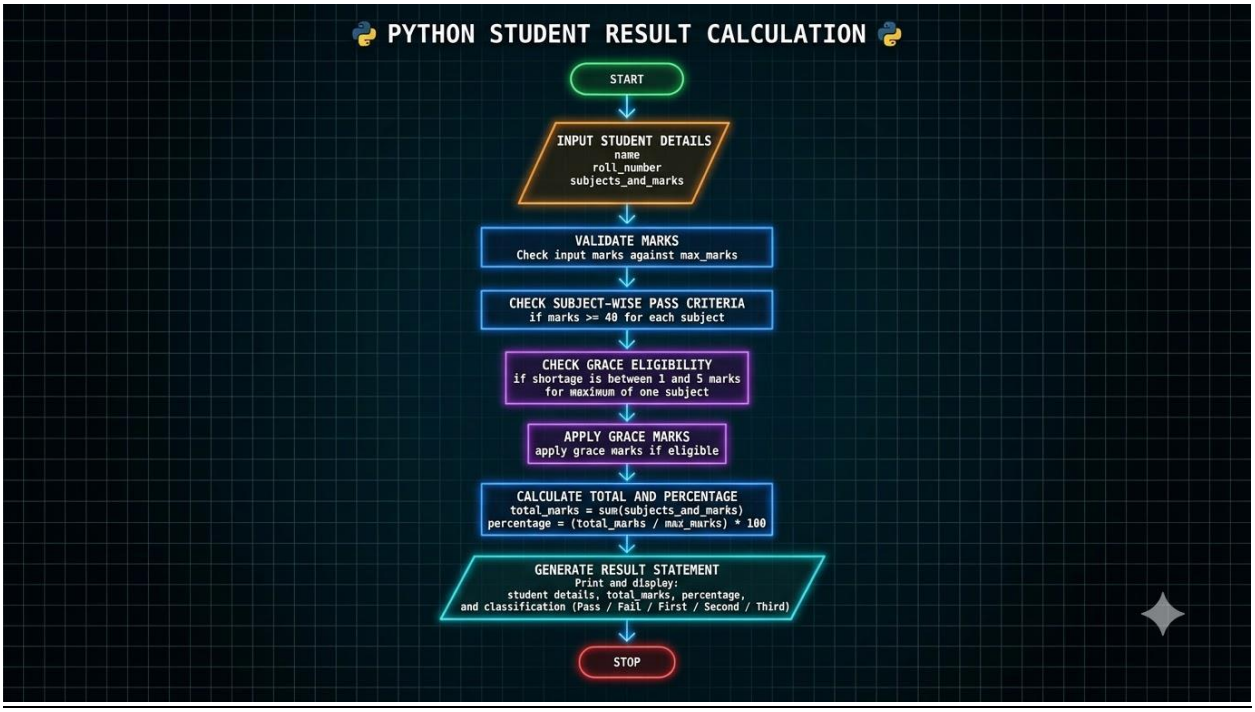
If a student scores 34, the student is not eligible because 6 marks are required.

3. Algorithm

1. Start.
2. Read student name and roll number.
3. Read the number of subjects.
4. Read subject names and marks.
5. Check whether entered marks are valid.
6. Store subject details in a nested list.
7. Check each subject's marks.
8. If marks are below 40, calculate the shortage.

9. If shortage is between 1 and 5 and no other subject has received grace marks, award grace marks.
10. Update final marks without exceeding maximum marks.
11. Calculate total marks.
12. Calculate percentage.
13. Check final pass or fail status.
14. Assign classification.
15. Display the examination result.
16. Stop.

4. Flowchart



---

5. ps**eudocode**

START

INPUT student_name

INPUT roll_number

```
INPUT number_of_subjects

pass_marks = 40
grace_limit = 5
grace_used = 0

FOR each subject:

    INPUT subject_name
    INPUT marks
    INPUT maximum_marks

    IF marks < 0 OR marks > maximum_marks:
        DISPLAY "Invalid Marks"

    ELSE:
        IF marks < pass_marks:

            shortage = pass_marks - marks

            IF shortage <= grace_limit AND grace_used == 0:

                grace = shortage
                marks = marks + grace
                grace_used = 1
```

```
            ELSE:
                grace = 0

        ELSE:
            grace = 0

        STORE subject details

CALCULATE total_marks
CALCULATE maximum_total
CALCULATE percentage

IF any subject is below pass_marks:
    result = "FAIL"

ELSE:
    result = "PASS"

IF result == "PASS":

    IF percentage >= 75:
        classification = "DISTINCTION"

    ELSE IF percentage >= 60:
        classification = "FIRST CLASS"
```

ELSE IF percentage >= 50:

classification = "SECOND CLASS"

ELSE:

classification = "THIRD CLASS"

DISPLAY examination result

STOP

6. Complete Python Program

```
# Examination Result and Grace-Mark Validation

# Student details
student_name = input("Enter student name: ")
roll_number = input("Enter roll number: ")

number_of_subjects = int(input("Enter number of subjects: "))

# Examination rules
pass_marks = 40
grace_limit = 5
grace_used = 0

# Nested list to store subject details
# [Subject, Original Marks, Grace Marks, Final Marks, Maximum Marks]
```

```
subjects = []

# Input and validation
for i in range(number_of_subjects):

    print("\nEnter details for Subject", i + 1)

    subject_name = input("Enter subject name: ")
    marks = int(input("Enter marks obtained: "))
    maximum_marks = int(input("Enter maximum marks: "))

    # Validate maximum marks and obtained marks
    if maximum_marks <= 0 or marks < 0 or marks > maximum_marks:
        print("Invalid marks! Enter valid details.")
        continue

    original_marks = marks
    grace = 0

    # Subject-wise pass checking
    if marks < pass_marks:

        shortage = pass_marks - marks

        # Grace-mark eligibility
        if shortage <= grace_limit and grace_used == 0:
```

```
            grace = shortage
            marks = marks + grace
            grace_used = 1

    # Ensure final marks do not exceed maximum marks
    if marks > maximum_marks:
        marks = maximum_marks

    # Store subject details
    subject_details = [
        subject_name,
        original_marks,
        grace,
        marks,
        maximum_marks
    ]

    subjects.append(subject_details)


# Calculate total marks
total_marks = 0
maximum_total = 0

failed_subjects = 0
```

```
total_grace = 0

for subject in subjects:

    # Tuple unpacking
    name, original, grace, final, maximum = (
        subject[0],
        subject[1],
        subject[2],
        subject[3],
        subject[4]
    )

    total_marks = total_marks + final
    maximum_total = maximum_total + maximum
    total_grace = total_grace + grace

    # Final subject pass checking
    if final < pass_marks:
        failed_subjects = failed_subjects + 1



# Calculate percentage
if maximum_total > 0:
    percentage = (total_marks / maximum_total) * 100
else:
```

```
    percentage = 0



# Generate final result
if len(subjects) != number_of_subjects:
    result = "INVALID / INCOMPLETE RESULT"

elif failed_subjects > 0:
    result = "FAIL"

else:
    result = "PASS"



# Classification
if result == "PASS":

    if percentage >= 75:
        classification = "DISTINCTION"

    elif percentage >= 60:
        classification = "FIRST CLASS"

    elif percentage >= 50:
        classification = "SECOND CLASS"
```

```
    else:
        classification = "THIRD CLASS"

else:
    classification = "NO CLASSIFICATION"



# Display final examination statement
print("\n==============================================")
print("         EXAMINATION RESULT")
print("==============================================")

print("Student Name :", student_name)
print("Roll Number  :", roll_number)

print("----------------------------------------------")
print("Subject     Original  Grace  Final  Maximum")
print("----------------------------------------------")

for subject in subjects:

    name, original, grace, final, maximum = (
        subject[0],
        subject[1],
        subject[2],
        subject[3],
```

```
        subject[4]
    )

    print(name, original, grace, final, maximum)

print("-------------------------------------------")

print("Total Marks     :", total_marks)
print("Maximum Marks   :", maximum_total)
print("Grace Marks     :", total_grace)
print("Percentage      :", round(percentage, 2), "%")
print("Final Result    :", result)
print("Classification  :", classification)

print("===========================================")
print("       RESULT PROCESSING COMPLETED")
```

7. Sample Input

Enter student name: Rahul

Enter roll number: 101

Enter number of subjects: 5

Enter details for Subject 1

Enter subject name: Python

Enter marks obtained: 85

Enter maximum marks: 100

Enter details for Subject 2

Enter subject name: Maths

Enter marks obtained: 38

Enter maximum marks: 100

Enter details for Subject 3

Enter subject name: English

Enter marks obtained: 75

Enter maximum marks: 100

Enter details for Subject 4

Enter subject name: Physics

Enter marks obtained: 65

Enter maximum marks: 100

Enter details for Subject 5

Enter subject name: Chemistry

Enter marks obtained: 80

Enter maximum marks: 100

8. Sample Output

==========

EXAMINATION RESULT

==========

Student Name : Rahul

Roll Number : 101

Subject Original Grace Final Maximum

Python 85 0 85 100

Maths 38 2 40 100

English 75 0 75 100

Physics 65 0 65 100

Chemistry 80 0 80 100

Total Marks : 345

Maximum Marks : 500

Grace Marks : 2

Percentage : 69.0 %

Final Result : PASS

Classification : FIRST CLASS

==========

RESULT PROCESSING COMPLETED

9. Classification Rules

Percentage	**Classification**
75% and above	Distinction
60%–74.99%	First Class

50%–59.99%	Second Class
40%–49.99%	Third Class
Below 40%	Fail

A student must pass every subject after applying the grace-mark policy to receive a classification.

10. Assigned Python Syllabus Concepts Used

Syllabus concept	**Application**
Computational thinking	Breaking examination processing into steps
Algorithm	Step-by-step solution
Flowchart	Program execution flow
Pseudocode	Logical representation
Variables	Marks, percentage, grace marks
Data types	String, integer, float
Operators	+, -, *, /, <, >
Conditional statements	if, elif, else
Loops	for loop
Strings	Student and subject names
Lists	Store subject information
Nested lists	Store multiple subject records
Tuples	Store unpacked subject details
Tuple unpacking	Extract subject information

Important: This is an illustrative grace-mark policy, not an official university examination regulation. The program assumes a pass mark of 40 for each subject and a maximum of one grace-mark award.

Complete syllabus-based solution